



Transportation's Systems 20-5751

PROJECT PART 2: INCREMENTAL ASSIGNMENT

PROF. ZAHRA AMINI
KEIVAN JAMALI

1405/01/23

1 Introduction to Incremental Assignment

Incremental Assignment (IA) is an iterative traffic assignment method that refines the AON assignment by gradually increasing traffic volumes and updating link travel times based on these volumes. Unlike AON, which uses fixed travel times, IA acknowledges that congestion forms as demand is added to the network. This leads to a more realistic distribution of traffic, as drivers tend to avoid routes that become heavily congested.

The IA method works by performing a series of AON assignments, each with a fraction of the total demand. After each AON assignment, the link travel times are updated using a user-defined travel time function (e.g., the BPR function), and the next fraction of demand is assigned based on these updated times. This process continues until all demand has been assigned.

2 Travel Time Functions and Demand

For Part 2, we will use the same Sioux Falls network, the same student-generated fixed link travel times from Part 1, and the same student-generated O-D demand. However, we will now utilize the full travel time function to simulate congestion.

Link Travel Time Functions (BPR Function): Recall the BPR function introduced conceptually in Part 1. We will now use it to update travel times dynamically. For each link (i, j) :

$$T_{ij}(f_{ij}) = T_{ij}^0 \left(1 + B \left(\frac{f_{ij}}{C_{ij}} \right)^4 \right)$$

Where:

- $T_{ij}(f_{ij})$ is the travel time on link (i, j) as a function of flow f_{ij} .
- T_{ij}^0 is the free-flow travel time, calculated in Part 1 as $A \times L_{ij}$.
- B is the parameter generated in Part 1.
- f_{ij} is the current flow on link (i, j) .
- C_{ij} is the capacity of link (i, j) . For this assignment, we assume a uniform capacity $C_{ij} = 1000$ vehicles per hour for all links.

Travel Demand: The total O-D demand D_{od} generated in Part 1 will be used.

3 The Incremental Assignment Algorithm

The Incremental Assignment process will involve iteratively assigning fractions of the total demand. A common approach is to divide the total demand into a fixed number of increments (e.g., 10 increments).

Algorithm Steps:

1. Initialization:

- Load the Sioux Falls network graph with link lengths L_{ij} and free-flow travel times T_{ij}^0 (calculated using parameter A) as edge attributes.
- Regenerate or retrieve parameter B for each link using the method from Part 1.
- Set link capacities $C_{ij} = 1000$ for all links.
- Initialize flow $f_{ij} = 0$ for all links.
- Generate the total O-D demand matrix D_{od} as done in Part 1.
- Define the number of increments, N , e.g., $N = 10$. The demand increment for each step will be $\Delta D_{od} = D_{od}/N$.

2. Iteration Loop: Repeat for $k = 1, 2, \dots, N$:

- (a) **Update Travel Times:** For each link (i, j) , calculate the current travel time T_{ij} using the BPR function with the current flow f_{ij} :

$$T_{ij} = T_{ij}^0 \left(1 + B \left(\frac{f_{ij}}{C_{ij}} \right)^4 \right)$$

Update the edge attribute for travel time in the graph.

- (b) **Perform AON Assignment for Increment:** For each O-D pair (o, d) with demand D_{od} :
- Find the shortest path from o to d using the **current** link travel times T_{ij} .
 - Assign the increment of demand $\Delta D_{od} = D_{od}/N$ to all links along this shortest path. Add this ΔD_{od} to the existing flow f_{ij} for each link on the path.

- 3. Final Flows:** After N iterations, the f_{ij} values represent the final link flows resulting from the incremental assignment.

4 Output Requirements

1. Use $n = 20$ and assign the traffic flows(Each step you should consider 100/20 percent = 5%). This is the first output.
2. Similare to Homework 2, assign the flow by 10 percent until a self defined criteria met(it will be more than 50 steps to converge). This will be the second output.

And submit the following:

1. **Python Script:** The complete, runnable Python script that performs all steps: network loading, travel time generation, demand generation, IA assignment, and calculation of the final checkable number.
2. **Small Report:** A brief report (e.g., a plain text file or a short markdown document) containing:
 - Your `student_id`.
 - The parameters A and B generated for three arbitrarily chosen links, showing they were derived using the specified `hash()` method.
 - Resultant network with the flows you calculated in this task.
3. **IMPORTANT__Output:** A single txt file for each output that summarizes the IA assignment results.
 - A text file that each line shows a link in this format:
1, 2, 150.5
2, 1, 44.3
...
 - Pay attention, the seprator between each item in line is ", " and the flow is rounded to 1 decimal.